

Reporte de Práctica: Instalación y Uso de GitHub CLI

Universidad Veracruzana Facultad de Ingeniería de la Computación

- **Estudiante:** Andrea García Ramos
- **Experiencia Educativa:** Documentación de Proyectos de Tecnologías Computacionales
- **Fecha:** Mayo 2026

1. Introducción

GitHub CLI (**gh**) es una herramienta de línea de comandos que permite administrar repositorios de GitHub directamente desde la terminal, reduciendo la necesidad de utilizar el navegador web.

1.1 Ventajas del uso de GitHub CLI

- Permite trabajar completamente desde la terminal.
- Facilita la automatización de tareas.
- Integra comandos de Git y GitHub en un mismo entorno.
- Mejora el flujo de trabajo en proyectos colaborativos.

2. Desarrollo de la Práctica y Proceso de Configuración

En esta sección detallo el proceso paso a paso que seguí para la instalación, configuración del entorno seguro y las pruebas de funcionamiento de la herramienta GitHub CLI (**gh**) en mi equipo local.

2.1 Instalación del Cliente y Configuración del PATH

Para iniciar con la práctica, realicé la instalación de la herramienta en mi sistema operativo Windows. Opté por utilizar el gestor de paquetes de Windows desde la terminal de Git Bash ejecutando el siguiente comando:

```
winget install --id GitHub.cli
```

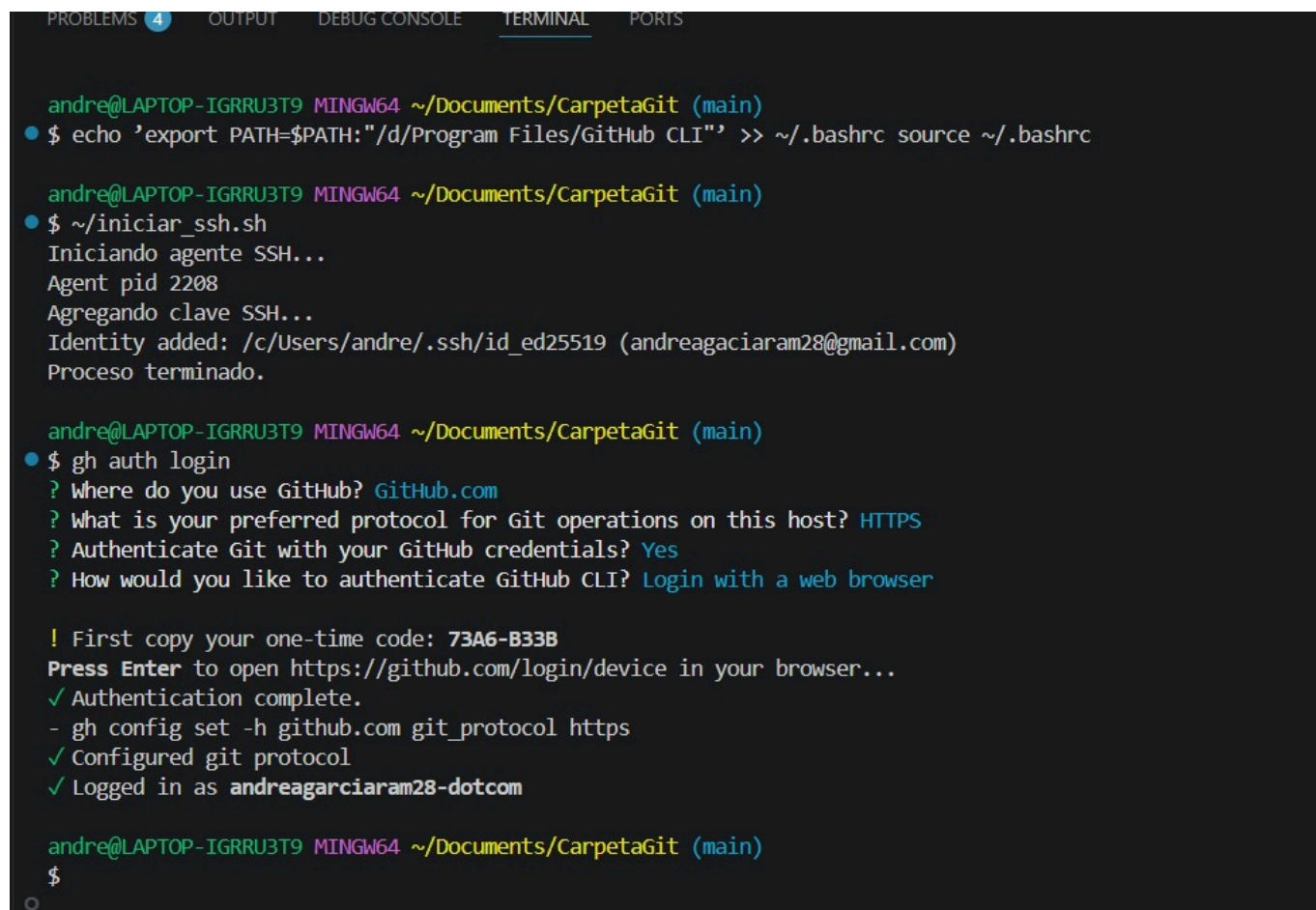
Una vez finalizada la instalación, procedí a verificar si el comando ya se encontraba disponible globalmente en el sistema con:

```
gh --version
```

```
andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetGit (main)
• $ gh --version
gh version 2.92.0 (2026-04-28)
https://github.com/cli/cli/releases/tag/v2.92.0
```

Al principio, la terminal de Git Bash no reconocía el comando gh. Esto se debió a que la ruta del ejecutable no se había indexado automáticamente en las variables de entorno del sistema. Para solucionar este inconveniente de forma inmediata, agregué manualmente la ruta de instalación al PATH del entorno actual usando:

```
export PATH=$PATH:"/d/Program Files/GitHub CLI"
```



```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ echo 'export PATH=$PATH:"/d/Program Files/GitHub CLI"' >> ~/.bashrc source ~/.bashrc

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ ~/iniciar_ssh.sh
Iniciando agente SSH...
Agent pid 2208
Agregando clave SSH...
Identity added: /c/Users/andre/.ssh/id_ed25519 (andreagaciar28@gmail.com)
Proceso terminado.

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 73A6-B33B
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as andreagaciar28-dotcom

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
$
```

Para evitar tener que ejecutar este comando cada vez que abriera una nueva terminal, decidí guardar la configuración de forma permanente. Para ello, procedí a exportar la ruta directamente al archivo de configuración de mi perfil de Git Bash (.bashrc) mediante el siguiente comando:

```
echo 'export PATH=$PATH:"/d/Program Files/GitHub CLI"' >> ~/.bashrc
```

Finalmente, apliqué los cambios en la sesión activa con `source ~/.bashrc` y volví a comprobar la versión, confirmando que el comando `gh` ya respondía de manera correcta y permanente.

2.2 Configuración del Entorno Seguro (Agente SSH)

Debido a que decidí trabajar las conexiones hacia GitHub mediante llaves criptográficas SSH para mayor seguridad, fue necesario levantar el servicio del agente de autenticación en la terminal.

Para automatizar este paso, ejecuté el script de inicialización que tengo configurado en mi directorio local:

```
~/iniciar_ssh.sh
```

```
andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 73A6-B33B
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as andreagarciam28-dotcom
```

Al ejecutarse el script, la terminal me devolvió el identificador del proceso (Agent pid), confirmando que el agente de SSH se estaba ejecutando en segundo plano de manera correcta. Acto seguido, procedí a registrar mi llave privada en el agente utilizando el comando:

```
ssh-add
```

La terminal me solicitó la frase de paso (passphrase) de mi llave criptográfica; la introduje correctamente y el agente confirmó que la identidad fue agregada con éxito.

2.3 Proceso de Autenticación en GitHub CLI

Con el PATH configurado y el agente SSH activo, procedí a vincular mi cuenta de GitHub con la herramienta de línea de comandos. Para iniciar este proceso interactivo, ejecuté:

```
gh auth login
```

```
andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ gh repo create repositorioclase-12mayo --public
✓ Created repository andreagarciam28-dotcom/repositorioclase-12mayo on github.com
https://github.com/andreagarciam28-dotcom/repositorioclase-12mayo

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ gh repo list

Showing 6 of 6 repositories in @andreagarciam28-dotcom
```

NAME	DESCRIPTION	INFO	UPDATED
andreagarciam28-dotcom/repositorioclase-12mayo		public	less than a minute ago
andreagarciam28-dotcom/Muros	proyecto de software	public, fork	about 6 days ago
andreagarciam28-dotcom/Repositoriofunc2		public	about 13 days ago
andreagarciam28-dotcom/Repositoriofunc		public	about 21 days ago
andreagarciam28-dotcom/Repositorio2		public	about 22 days ago
andreagarciam28-dotcom/Repositorio1		public	about 22 days ago

```
andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
o $
```

A partir de ahí, fui seleccionando las opciones que me solicitaba la interfaz en la terminal:

Account: Seleccioné GitHub.com.

Preferred protocol for Git operations: Elegí SSH para aprovechar la configuración del agente que preparé en el paso anterior.

Upload your SSH public key: Elegí asociar mi llave pública existente.

Title for SSH key: Le asigné un nombre identificable para mi perfil de GitHub.

Authentication method: Elegí realizar la autenticación a través del navegador web (Log in with a web browser).

La terminal me proporcionó un código único de 8 caracteres y abrió automáticamente el navegador. Introduje el código en la página de validación de GitHub, autoricé los permisos de la aplicación y, de inmediato, la terminal me mostró el mensaje de éxito confirmando que había iniciado sesión correctamente como Andrea Garcia Ramos.

2.4 Pruebas de Funcionamiento y Flujo de Trabajo

Para comprobar que la herramienta estaba bien enlazada y me permitía gestionar repositorios, realicé un flujo de trabajo completo desde la terminal:

Creación del repositorio: Creé un nuevo repositorio remoto y público directamente en mi perfil de GitHub con el comando:

```
gh repo create repositorioclase-12mayo --public
```

```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ echo 'export PATH=$PATH:"/d/Program Files/GitHub CLI"' >> ~/.bashrc source ~/.bashrc

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ ~/iniciar_ssh.sh
Iniciando agente SSH...
Agent pid 2208
Agregando clave SSH...
Identity added: /c/Users/andre/.ssh/id_ed25519 (andreagaciar28@gmail.com)
Proceso terminado.

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
• $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 73A6-B33B
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as andreagaciar28-dotcom

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetaGit (main)
$

```

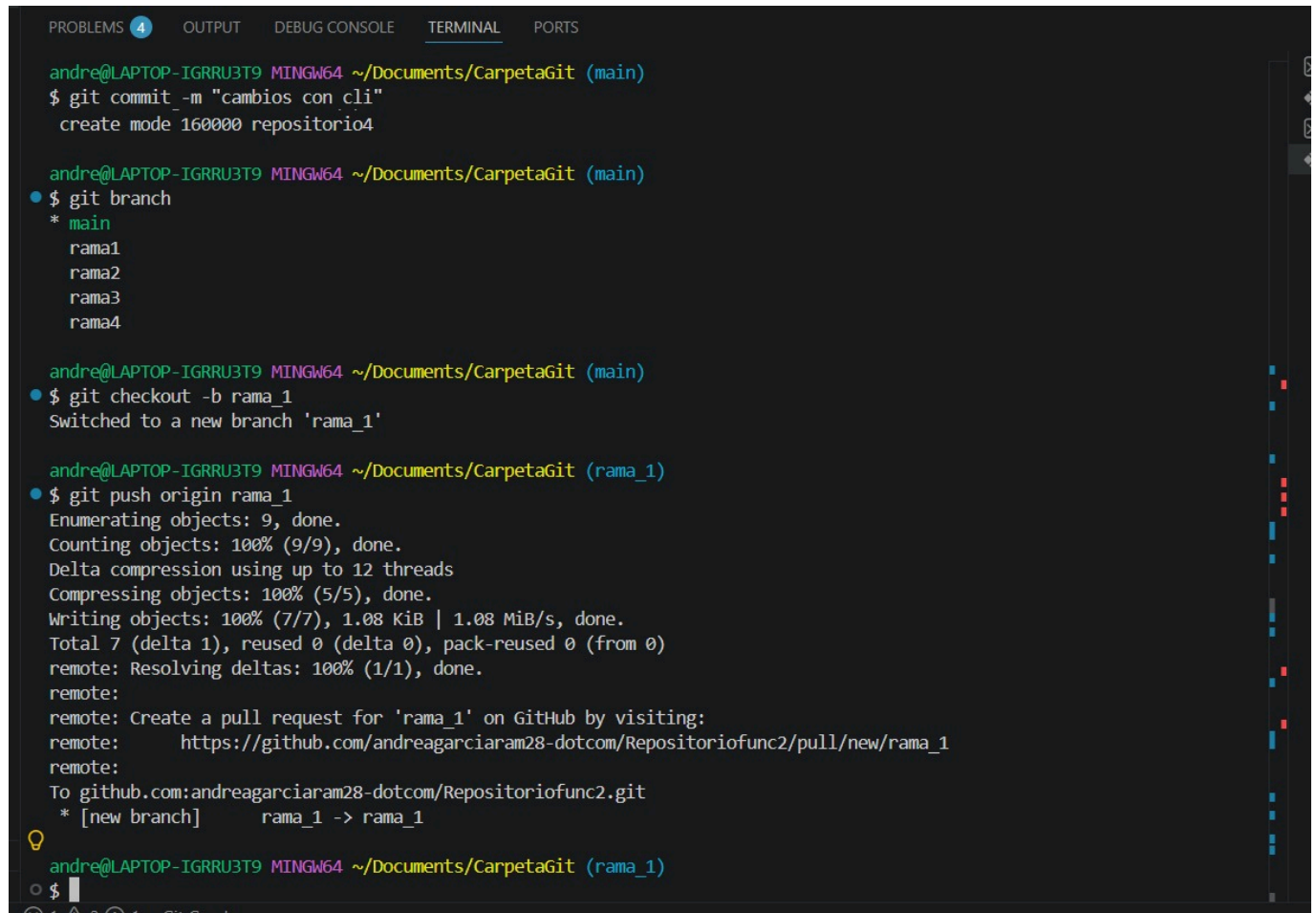
Inicialización local y enlace: Inicialicé un repositorio Git local en mi máquina, creé un archivo inicial, agregué el origen remoto apuntando a la URL SSH de mi nuevo repositorio y subí los cambios iniciales a la rama principal (main).

Creación de ramas paralelas: Para simular un entorno de trabajo colaborativo o de desarrollo de características, creé una nueva rama de trabajo llamada rama_1 mediante los comandos tradicionales de Git:

```
git checkout -b rama_1
```

Realicé modificaciones en los archivos de la práctica, generé un commit de confirmación y subí la rama al servidor remoto con:

```
git push origin rama_1
```



```

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetGit (main)
$ git commit -m "cambios con cli"
create mode 160000 repositorio4

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetGit (main)
$ git branch
* main
  rama1
  rama2
  rama3
  rama4

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetGit (main)
$ git checkout -b rama_1
Switched to a new branch 'rama_1'

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetGit (rama_1)
$ git push origin rama_1
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 1.08 KiB | 1.08 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'rama_1' on GitHub by visiting:
remote:   https://github.com/andreagarciam28-dotcom/Repositoriofunc2/pull/new/rama_1
remote:
To github.com:andreagarciam28-dotcom/Repositoriofunc2.git
 * [new branch]      rama_1 -> rama_1

andre@LAPTOP-IGRRU3T9 MINGW64 ~/Documents/CarpetGit (rama_1)
$

```

Gestión de Pull Requests mediante la CLI: En lugar de entrar a la página web de GitHub para proponer la integración de mis cambios, utilicé la CLI para abrir un Pull Request de manera interactiva:

```
gh pr create
```

A través de los prompts de la terminal, le asigné el título "rama 1" al Pull Request y añadí una descripción sobre los cambios realizados. La herramienta procesó la solicitud y me devolvió la URL final del Pull Request creado en el servidor, demostrando la alta eficiencia que ofrece el uso de GitHub CLI para optimizar los flujos de desarrollo.

2. Documentación de Comandos de GitHub CLI

Comando 1: **gh repo clone**

- **Descripción:** Permite clonar un repositorio alojado en GitHub en la máquina local.
- **Propósito/Utilidad:** Descargar proyectos remotos de manera rápida sin salir de la terminal.

- **Sintaxis:** `gh repo clone <usuario/repositorio>`

- **Ejemplo práctico:**

```
gh repo clone lolmedo-99/proyecto-demo
```

Comando 1: `gh auth login`

Descripción: Inicia el proceso de autenticación en GitHub.

Utilidad: Permite conectar la terminal local con tu cuenta remota de manera segura.

Sintaxis: `gh auth login [flags]`

Ejemplo práctico: `Bashgh auth login`

Comando 2: `gh repo create`

Descripción: Crea un nuevo repositorio en GitHub.

Utilidad: Permite inicializar proyectos remotos directamente sin abrir el navegador web.

Sintaxis: `gh repo create [flags]` Ejemplo práctico: `Bashgh repo create repositorioclase-12mayo --public`

Comando 3: `gh repo clone`

Descripción: Permite clonar un repositorio alojado en GitHub. Utilidad: Descarga una copia completa de un proyecto remoto a la computadora local.

Sintaxis: `gh repo clone <usuario/repositorio>`

Ejemplo práctico: `Bashgh repo clone lolmedo-99/repositorio2`

Comando 4: `gh repo list` Descripción: Muestra los repositorios asociados a la cuenta.

Utilidad: Visualizar rápidamente el listado de proyectos disponibles y su información básica. Sintaxis: `gh repo list [usuario]` Ejemplo práctico: `Bashgh repo list`

Comando 5: `gh repo delete` Descripción: Elimina un repositorio de GitHub.

Utilidad: Borrar proyectos de forma definitiva directamente desde la terminal.

Sintaxis: `gh repo delete <usuario/repositorio> [flags]`

Ejemplo práctico: `Bashgh repo delete Paola1426/Repositoriofork`

Comando 6: `gh repo fork` Descripción: Crea un fork de un repositorio.

Utilidad: Duplicar un repositorio ajeno en tu cuenta personal para poder trabajar en él.

Sintaxis: `gh repo fork [repositorio] [--fork-name nuevo_nombre]`

Ejemplo práctico: `Bashgh repo fork lolmedo-99/repositorio --fork-name nuevo_nombre`

Comando 7: `gh pr create` Descripción: Crea un Pull Request en GitHub.

Utilidad: Proponer los cambios realizados en tu rama para que se fusionen con la rama principal del proyecto.

Sintaxis: `gh pr create [flags]`

Ejemplo práctico: `Bashgh pr create --title "rama 1" --body "Descripción de los cambios"`

Comando 8: `gh auth status` Descripción: Muestra el estado actual de la autenticación.

Utilidad: Verificar qué cuenta está activa en la CLI y qué permisos tiene configurados. Sintaxis: `gh auth status [flags]` Ejemplo práctico: `Bashgh auth status`

Comando 9: `gh auth refresh` Descripción: Actualiza los alcances (scopes) de la sesión activa.

Utilidad: Añadir permisos adicionales sobre la marcha (como permisos para borrar repositorios). Sintaxis: `gh`

`auth refresh -h -s`

Ejemplo práctico: `Bashgh auth refresh -h github.com -s delete_repo`

Comando 10: `gh issue list` Descripción: Lista las incidencias (issues) de un repositorio.

Utilidad: Ver de forma rápida las tareas pendientes o errores reportados en el proyecto.

Sintaxis: `gh issue list [flags]`

Ejemplo práctico: `Bashgh issue list`

Comando 11: `gh issue create` Descripción: Crea una nueva incidencia.

Utilidad: Reportar un bug o proponer una mejora en el repositorio sin abrir la web.

Sintaxis: `gh issue create [flags]` Ejemplo práctico: `Bashgh issue create --title "Error en base de datos" --body "No conecta con MariaDB"`

Comando 12: `gh --help` Descripción: Muestra la documentación integrada en la terminal.

Utilidad: Consultar la guía jerárquica de comandos, subcomandos y banderas disponibles de la herramienta.

Sintaxis: `gh [comando] --help`

Ejemplo práctico: `Bashgh repo clone --help`

4. Conclusión

A lo largo del desarrollo de esta práctica, logré comprobar que la implementación de interfaces de línea de comandos especializadas, como GitHub CLI ([gh](#)), representa una optimización crítica en el flujo de trabajo de un desarrollador de software. Al unificar los procesos tradicionales de control de versiones distribuidas (Git) con las capacidades de gestión remota de la plataforma de alojamiento (GitHub), se disminuye sustancialmente el contexto de cambio físico entre la terminal de comandos y el navegador web. Esto no solo se traduce en un incremento en la velocidad de ejecución y automatización de tareas, sino que también minimiza los puntos de distracción visual durante las fases de desarrollo intensivo.

Asimismo, la integración de protocolos criptográficos seguros mediante llaves SSH y agentes locales demostró ser una solución eficiente para resguardar la identidad digital del usuario en entornos colaborativos, eliminando la necesidad de interactuar constantemente con credenciales vulnerables de texto plano. Por otra parte, la documentación de la práctica utilizando el lenguaje de marcado Markdown puso de manifiesto el valor de emplear estándares ligeros y universales para la estructuración de manuales técnicos, facilitando su posterior portabilidad y exportación hacia formatos estandarizados de distribución como el PDF. En definitiva, el dominio conjunto de la terminal de comandos, metodologías seguras de autenticación y documentación técnica estructurada constituye una competencia fundamental en la formación profesional dentro del área de las Tecnologías Computacionales.